Fault Tolerance

fault vs failure, zones and blast radius

Watch the 3-min video

In one line: faults (a disk dies, a bad deploy) are certain; fault tolerance stops them becoming failures (users affected) through redundancy, isolation and fast detection.

1 Fault vs failure

A **fault** is one component going wrong: a disk fails, a process crashes, a dependency slows down. A **failure** is the whole system no longer serving users. Fault tolerance means stopping faults from turning into failures.

Kind of fault	Examples
Hardware	Dead disks, bad memory, power or network loss
Software	Bugs, memory leaks, a dependency that slows to a crawl, a retry storm
Human	A wrong config, a bad deploy, a deleted table. In practice a large share of outages.

2 How to tolerate faults

- **Redundancy and replication:** more than one copy of every important thing.
- **Isolation:** a problem in one part can't spread to the rest.
- **Fast detection:** health checks, heartbeats and timeouts move traffic away from broken parts automatically.

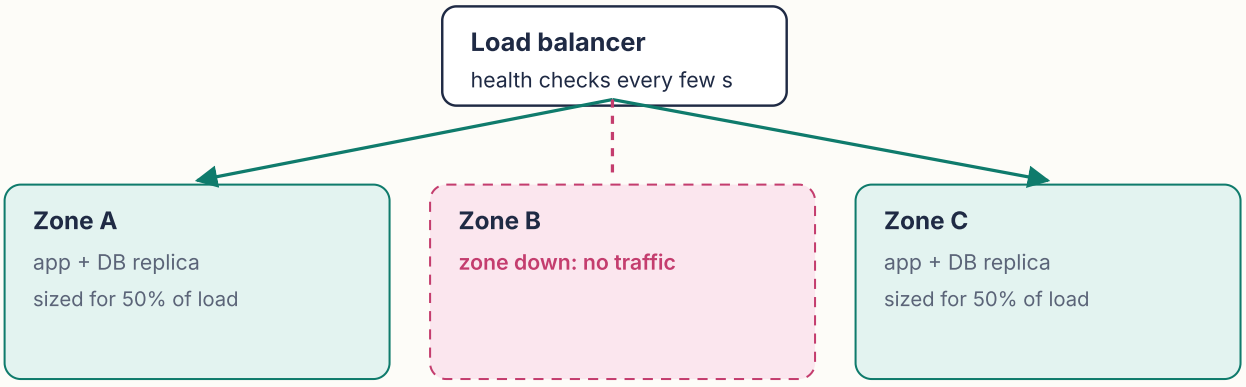


Figure 1. Replicas in three availability zones. If one zone goes down, the load balancer stops sending it traffic and the other two carry the load.

PLAN CAPACITY FOR A ZONE LOSS

With three zones at peak, each normally carries a third of the traffic. If one fails, the other two must carry half each. So each zone must be sized for ~50% of peak, about 1.5× its normal share. Without that headroom, losing one zone overloads the other two and turns a fault into a failure.

3 Blast radius

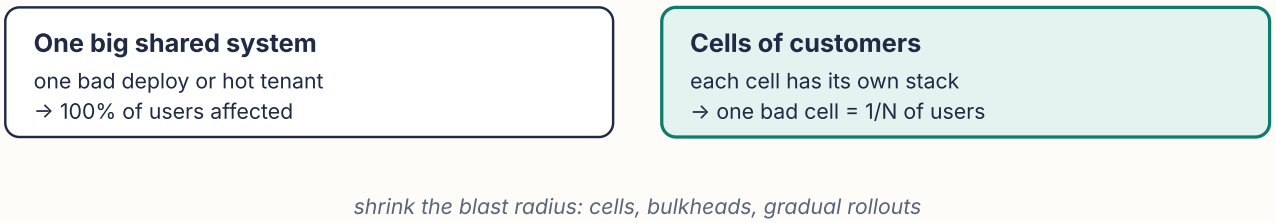


Figure 2. Cells limit how many users one problem can reach.

- **Cells or shards of customers:** each slice of users gets its own stack; a bad cell affects only its slice.
- **Bulkheads:** separate thread or connection pools per dependency, so one slow dependency can't use up all the workers.
- **Gradual rollouts:** canary to 1% of traffic, watch the metrics, then widen, with automatic rollback.

THE MINDSET: DESIGN FOR FAILURE

Assume every component will fail. For each box ask “what happens when this dies?”, then test it on purpose with chaos experiments (kill an instance, cut a zone, slow a dependency) before reality does.

COMMON MISTAKES

- Three zones, each sized only for a third of peak.
- All replicas sharing one config or deploy that breaks them together.
- No timeouts, so one slow dependency hangs every request.
- Big-bang deploys to 100% of users.

4 Key terms

Fault: one component misbehaves

Failure: users are affected

Availability zone: separate data centre in a region

Blast radius: how much breaks when something fails

Bulkhead: isolated pool per dependency

Chaos engineering: breaking things on purpose to learn

5 Interview questions

Q1. What's the difference between a fault and a failure? BASIC

A fault is one component misbehaving; a failure is the system no longer serving users. Fault tolerance stops faults from becoming failures.

Q2. How do you limit blast radius? BASIC

Isolate with cells and bulkheads, spread across zones, and roll out changes gradually with canaries and automatic rollback.

Q3. What are the three kinds of faults? BASIC

Hardware (disks, power, network), software (bugs, leaks, slow dependencies) and human (bad configs and deploys).

Q4. You run in 3 zones. How much capacity does each need? SCENARIO

Enough to survive losing one: each zone sized for about half of peak, so the remaining two can carry everything.

Q5. A slow payment provider makes your whole API time out. Fix? SCENARIO

Timeouts on the call, a bulkhead (separate pool) for the payment client, a circuit breaker that fails fast, and a fallback such as queueing the payment for retry.

Q6. A config push took down every server at once. How do you prevent it? SCENARIO

Treat config like code: validate it, roll it out gradually (one cell or 1% first), watch error rates, and roll back automatically.

Q7. Do more replicas always mean more fault tolerance? TRICKY

Only against independent faults. Replicas that share a bug, config, zone or dependency fail together; diversity and isolation matter as much as count.

Q8. Why run chaos experiments in production? TRICKY

Staging rarely matches production's traffic, data and dependencies. Small, controlled experiments in production (with a kill switch) reveal real weaknesses before an outage does.

Next: SD-A-017 · Single Point of Failure backendroadmap.com